Program Studi Teknik Informatika
Institut Teknologi Sumatera

IF25-11001 ALGORITMA PEMROGRAMAN

Modul Belajar 9

Array Satu Dimensi

Dari 40 nilai kuis satu kelas sampai alamat memori setiap elemennya

Tim Pengajar

Algoritma Pemrograman. Tahun 2026

Komponen	Keterangan
Pertemuan	9 dari 16
Capaian	Sub-CPMK 9.1 dan 9.2, turunan CPMK 0607 dan 0608
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 100 menit di kelas
Latihan	`02 Latihan/P9 - Array Satu Dimensi - Latihan/`

Peta belajar

Bagian	Isi
0	Tentang modul ini dan cara memakainya
1	Masalah pembuka
Langkah 1	array, indeks, dan deklarasi
Langkah 2	indeks dimulai dari nol
Langkah 3	alamat elemen di memori
Langkah 4	pola traversal
Langkah 5	trace table untuk akumulasi
Langkah 6	inisialisasi maksimum
Langkah 7	maksimum dan indeksnya
Langkah 8	menghapus dan menyisipkan
Langkah 9	dua kesalahan indeks yang paling sering
Penutup	Latihan mandiri, bawa ke tugas, rangkuman, cek pemahaman, bacaan lanjut

Tentang modul ini

Modul ini menemani Pertemuan 9. Isinya sama dengan slide, tetapi ditulis lebih pelan supaya bisa dipelajari sendiri, termasuk oleh yang berhalangan hadir.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Diukur di
9.1	Merancang pseudocode yang menyimpan data sejenis di array satu dimensi dan memprosesnya dengan traversal: total, rata-rata, maksimum beserta indeksnya, dan counting	Exit ticket soal 1, Tugas 9, PBL 2, UAS
9.2	Menelusuri isi array dengan trace table, menghitung alamat elemen, dan menjelaskan kesalahan indeks termasuk akses di luar batas	Exit ticket soal 2, Tugas 9, UAS

Keduanya menurunkan CPMK 0607 dan 0608: menerapkan (0607) dan menjelaskan (0608) konsep dasar algoritma dan sistem komputer untuk menyelesaikan masalah komputasi.

Cara memakai modul ini

Setiap langkah punya kotak merah **Tebak dulu**. Tutup kotak hijau **Jawaban** di bawahnya dengan kertas, tulis dugaan kalian, baru buka. Prosedur penting dijabarkan dalam tiga kolom berdampingan yang dibaca dari kiri ke kanan; sebelum membaca kolom berikutnya, coba tebak dulu langkah apa yang akan datang. Rangkaian setengah jadi lalu mandiri mengurangi bantuan sedikit demi sedikit.

Yang perlu disiapkan

Kertas bergaris, pensil, dan nilai *A* dan *B* dari NIM kalian: *A* digit terakhir (kalau 0, pakai 5) dan *B* digit kedua dari belakang (kalau 0, pakai 3).

Masalah pembuka

PEMBUKA

Seorang asisten memegang nilai kuis 40 mahasiswa. Ia perlu rata-rata kelas, nilai tertinggi beserta pemiliknya, dan banyak mahasiswa yang harus remedial. Dengan variabel biasa, setiap nilai butuh nama sendiri, dari `n1` sampai `n40`, dan mencari nilai tertinggi butuh 39 baris JIKA. Perulangan tidak bisa membantu karena namanya berbeda-beda. Yang dibutuhkan adalah satu nama dengan nomor urut, sehingga perulangan bisa berjalan dari nomor pertama sampai terakhir. Itulah array.

Langkah 1: array, indeks, dan deklarasi

ARRAY DAN INDEKS

Array adalah sederet kotak bertipe sama yang memakai satu nama. Kotak-kotak itu dibedakan oleh nomor yang disebut indeks, dan indeks pertama adalah 0. Deklarasi `DEKLARASI nilai[8] : integer` memesan delapan kotak sekaligus, dari `nilai[0]` sampai `nilai[7]`. Karena indeks boleh berupa variabel, satu perulangan bisa mengunjungi semua kotak: inilah yang tidak mungkin dilakukan dengan variabel n1 sampai n40. Ada tiga sifat array. Semua elemen bertipe sama, ukuran array tetap sejak dideklarasikan, dan elemen-elemennya disimpan bersebelahan di memori.

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
nilai	72	85	64	90	58	77	85	69

Langkah 2: indeks dimulai dari nol

ARRAY DAN INDEKS

Karena indeks dimulai dari 0, elemen ke-*k* disimpan di indeks *k* − 1. Kesalahan ini disebut *off-by-one*, kesalahan array yang paling sering muncul di tugas.

TEBAK DULU

Array `nilai[8]` berisi delapan nilai. Elemen kedelapan disimpan di `nilai[?]`. Lalu apa isi `nilai[8]`?

JAWABAN

Elemen kedelapan ada di `nilai[7]`. Kotak `nilai[8]` tidak pernah dipesan; membacanya berarti membaca memori milik variabel lain.

Langkah 3: alamat elemen di memori

ARRAY DAN INDEKS

Elemen array disimpan bersebelahan. Jika alamat elemen pertama (base) dan ukuran satu elemen diketahui, alamat elemen berindeks *i* adalah base + *i* × ukuran. Rumus ini sekaligus menjelaskan mengapa indeks dimulai dari 0: indeks adalah jarak elemen itu dari awal array. Contohnya, untuk integer 4 byte dengan base 1000, alamat `nilai[5]` adalah 1000 + 5 × 4 = 1020.

1 Bersebelahan

Kotak berikutnya mulai tepat setelah kotak sebelumnya selesai:

`nilai[1]` di 1004, `nilai[2]` di 1008.

2 Rumus

alamat(*i*) = base + *i* × ukuran

Base adalah alamat `nilai[0]`, ukuran adalah byte per elemen.

3 Contoh

`nilai[5]`: 1000 + 5 × 4 = 1020.

Indeks ternyata adalah **jarak dari awal**, diukur dalam elemen. Elemen pertama berjarak 0.

Mengapa ini berguna

Komputer tidak perlu menelusuri dari depan untuk mengambil `nilai[5]`. Satu perkalian dan satu penjumlahan sudah cukup, berapa pun panjang arraynya.

UJI DIRI

Pilih jawaban yang benar. (A) 2024, (B) 2028, (C) 2006, (D) 2040.

JAWABAN: A

$2000 + 6 \times 4 = 2024$. Pilihan B menghitung seolah indeks mulai dari 1. Pilihan C lupa mengalikan dengan ukuran. Pilihan D adalah alamat tepat setelah elemen terakhir, $2000 + 10 \times 4$.

Langkah 4: pola traversal

TRAVERSAL DAN STATISTIK

Hampir semua pemrosesan array memakai kerangka yang sama: perulangan dari 0 sampai $N - 1$ dengan `nilai[i]` di badannya. Yang berganti hanya isi badan perulangan. Karena setiap elemen dikunjungi sekali, banyak langkah traversal sebanding dengan banyak elemen, dan ini ditulis $O(n)$.

```
UNTUK i ← 0 SAMPAI N-1
  READ nilai[i]
AKHIR UNTUK
```

Langkah 5: trace table untuk akumulasi

TRAVERSAL DAN STATISTIK

Trace table adalah tabel yang mencatat nilai setiap variabel setelah setiap putaran. Untuk data kuis 72, 85, 64, 90, 58, 77, 85, 69, total bertambah menjadi 72, 157, 221, 311, 369, 446, 531, lalu 600, sehingga rata-ratanya 75. Cara memeriksa trace table: nilai di kolom sebelum pada suatu baris harus sama dengan kolom sesudah di baris atasnya.

```
total ← 0
UNTUK i ← 0 SAMPAI N-1
  total ← total + nilai[i]
AKHIR UNTUK
rata ← total / N
```

Langkah 6: inisialisasi maksimum

TRAVERSAL DAN STATISTIK

Nilai awal akumulator menentukan benar tidaknya hasil. Total dan count wajar dimulai dari 0. Maksimum tidak, karena 0 mungkin lebih besar dari semua data.

TEBAK DULU

Suhu freezer laboratorium biologi selama lima hari: $-5, -3, -8, -1, -7$. Rina menulis `maks ← 0` lalu membandingkan setiap elemen dengan maks. Apa yang dicetak?

JAWABAN

Tercetak 0, nilai yang tidak ada di data, karena tidak ada elemen yang lebih besar dari 0. Mulailah dari `maks ← arr[0]`, lalu bandingkan mulai indeks 1.

Langkah 7: maksimum dan indeksnya

TRAVERSAL DAN STATISTIK

Untuk mengetahui posisi nilai tertinggi, simpan indeksnya bersama nilainya. Setiap kali ada elemen yang lebih besar, keduanya diganti dalam blok JIKA yang sama. Untuk data kuis, maks berganti dari 72 ke 85, lalu ke 90 saat $i = 3$, sehingga hasilnya 90 di indeks 3. Karena pembandingnya lebih dari (bukan lebih dari atau sama dengan), nilai kembar 85 yang tercatat adalah kemunculan pertamanya.

```
maks ← nilai[0]
posMaks ← 0
UNTUK i ← 1 SAMPAI N-1
  JIKA nilai[i] > maks MAKA
    maks ← nilai[i]
    posMaks ← i
  AKHIR JIKA
AKHIR UNTUK
```

Setengah jadi

Minimum beserta indeksnya.

```
mini ← ____
posMin ← ____
UNTUK i ← ____ SAMPAI N-1
  JIKA nilai[i] ____ mini MAKA
    mini ← nilai[i]
    ____ ← i
```

Mandiri

Tulis pseudocode yang menghitung berapa mahasiswa harus remedial, yaitu yang nilainya di bawah 60. Trace untuk data kuis.

JAWABAN

Setengah jadi: `nilai[0]`, `0`, `1`, `<`, `posMin`; hasilnya 58 di indeks 4. Mandiri: `count ← 0`, perulangan 0 sampai $N - 1$, `JIKA nilai[i] < 60 MAKA count ← count + 1`; hasilnya 1.

UJI DIRI

Pilih jawaban yang benar. (A) 2, (B) 4, (C) 5, (D) 172.

JAWABAN: A

Hanya 82 dan 90 yang lebih dari 75. Pilihan B menganggap > sama dengan $\geq$. Pilihan C mengembalikan banyak elemen. Pilihan D menjumlahkan nilainya, padahal counting menambah 1.

Langkah 8: menghapus dan menyisipkan

MENGGESER ISI ARRAY

Ukuran array tetap, sehingga menghapus elemen berindeks k dilakukan dengan menggeser semua elemen di kanannya satu langkah ke kiri, lalu mengurangi N. Menyisipkan di indeks k kebalikannya: elemen digeser ke kanan, tetapi harus dimulai dari belakang, supaya tidak ada nilai yang tertimpa sebelum disalin.

```
// sisip x di indeks k, kapasitas cukup
UNTUK i ← N-1 TURUN SAMPAI k
  arr[i+1] ← arr[i]
AKHIR UNTUK
arr[k] ← x
N ← N + 1
```

UJI DIRI

Pilih jawaban yang benar. (A) 4, 5, 7, 9, (B) 4, 5, 7, 7, (C) 4, 5, 9, _, (D) 5, 4, 7, 9.

JAWABAN: B

Saat $i = 1$, `arr[2]` menjadi 7 dan angka 9 hilang. Saat $i = 2$, `arr[3]` menyalin 7 itu. Pilihan A adalah yang diinginkan, bukan yang dikerjakan pseudocode-nya.

Langkah 9: dua kesalahan indeks yang paling sering

MENGGESER ISI ARRAY

Dua kesalahan indeks muncul berulang kali. Pertama, perulangan dari 1 sampai N: elemen indeks 0 terlewat dan `arr[N]` dibaca padahal tidak ada. Kedua, pergeseran yang membaca `arr[i+1]` sampai $i = N - 1$, sehingga `arr[N]` ikut dibaca. Pemeriksaan cepatnya: tulis indeks terbesar yang disentuh setiap baris, lalu pastikan tidak melebihi $N - 1$.

Latihan mandiri

Kerjakan lembar latihan di folder `02 Latihan`. Urutannya disusun dari mengingat ke menganalisis.

Tingkat	Waktu	Yang dilatih
1, Dasar	5′	membaca isi array dan menghitung alamat elemen
2, Menengah	8′	pseudocode total, rata-rata, minimum beserta indeksnya
3, Tantangan	7′	menemukan dan menjelaskan kesalahan indeks
Pengayaan	sisa	membalik array di tempat

Kesalahan yang sering muncul

Yang ditulis	Masalahnya	Obatnya
`UNTUK i ← 1 SAMPAI N`	`arr[0]` terlewat dan `arr[N]` dibaca	Batas 0 sampai $N - 1$
`maks ← 0`	Gagal jika semua data negatif	`maks ← arr[0]`, mulai indeks 1
Sisip dengan perulangan maju	Nilai tertimpa sebelum disalin	Geser dari belakang
`count ← count + arr[i]`	Itu menjumlah, bukan menghitung	`count ← count + 1`

Bawa ke tugas

Tugas 9 dikerjakan di kertas: untuk `arr` = 15, 3, 22, 8, 17, 11, 9, 20, tulis pseudocode dan trace table maksimum dan minimum beserta indeksnya, serta banyak elemen di atas rata-rata; hitung alamat elemen ke-0, 3, 7, dan 9 untuk `data[10]` bertipe double 8 byte dengan base 5000; tulis pseudocode membalik array di tempat. Untuk PBL 2, kelompok mulai merancang statistik dasar Data Analyzer dengan trace table minimal delapan elemen.

Rangkuman

Satu nama, banyak kotak bertipe sama, indeks 0 sampai $N - 1$, bersebelahan di memori: alamat = base + $i \times$ ukuran. Kerangka 0 sampai $N - 1$ tetap; badannya berganti. Total dan count mulai 0, maksimum mulai `arr[0]`. Indeks N tidak ada. Sisip digeser dari belakang, hapus dari depan, dan trace table menunjukkan apa yang benar-benar terjadi.

Cek pemahaman

Jawab tanpa membuka modul atau slide. Mengingat kembali tanpa melihat jauh lebih menguatkan ingatan daripada membaca ulang.

No	Soal
1	Array `x[12]`. Indeks elemen terakhir?
2	Base 4000, elemen 8 byte. Alamat `x[3]`?
3	Mengapa maksimum tidak dimulai dari 0?
4	Sisip di tengah: geser dari depan atau belakang?
5	Berapa putaran traversal array 40 elemen?

KUNCI CEK PEMAHAMAN

(1) 11. (2) $4000 + 3 \times 8 = 4024$. (3) Jika semua data negatif, 0 tidak pernah terkalahkan dan hasilnya salah. (4) Dari belakang, supaya tidak ada nilai yang tertimpa sebelum disalin. (5) 40, satu per elemen.

Bacaan lanjut

Gaddis, T. *Starting Out with C++*, Pearson. Deitel, P. dan Deitel, H. *C++ How to Program*, Pearson. Cormen dkk. *Introduction to Algorithms*, MIT Press. Visualisasi interaktif di `visualgo.net`.